Trabalho 01 (AP1) – Tradutor Dirigido por Sintaxe

Parte A – Projeto da tradução | Disciplina CC0032 – Compiladores | Semestre 2026.2

Aluno: Miguel Wagner Galvão Ferreira de Morais | Matrícula: 2023004265

Curso: Ciência da Computação – Universidade Federal do Cariri (UFCA)

Professor: Laszlon Rodrigues da Costa

Este relatório responde aos quatro itens da Parte A do enunciado. As árvores e todas as produções abaixo estão em texto editável.

1. Eliminação da recursão à esquerda

Enunciado: Aplique a técnica de eliminação da recursão à esquerda às produções de expr e term e escreva a gramática resultante.

Para uma produção A → Aα | Aβ | γ, aplica-se A → γR, R → αR | βR | ε. Em expr, α = + term, β = - term e γ = term. Em term, α = * fact, β = / fact e γ = fact.

```
expr → term R
R    → + term R | - term R | ε

term → fact T
T    → * fact T | / fact T | ε

fact → ( expr ) | num
```

Os auxiliares R e T representam, respectivamente, a continuação de somas/subtrações e de multiplicações/divisões. A produção fact é mantida, pois não possui recursão à esquerda. Conforme o enunciado, num é entregue como um token único pelo procedimento proximo; a regra lexical para vários dígitos não é implementada como função recursiva do analisador sintático.

2. Ações semânticas para gerar pós-fixa

Enunciado: Acrescente às produções do item 1 ações semânticas na posição exata em que devem ser executadas para produzir notação pós-fixada.

```
expr → term R

R → + term { emitir("+") } R
  | - term { emitir("-") } R
  | ε

term → fact T

T → * fact { emitir("*") } T
  | / fact { emitir("/") } T
  | ε

fact → ( expr )
     | num { emitir(num.valor) }
```

A função emitir representa a inclusão de um token na saída (com espaço entre tokens). A ação do operador vem depois de seu operando à direita e antes do R/T seguinte, preservando associatividade à esquerda. Os parênteses apenas delimitam uma subexpressão e não são emitidos.

3. Conjuntos FIRST e justificativa da análise preditiva

Enunciado: Calcule FIRST para cada alternativa dos não-terminais e justifique o uso de um analisador preditivo sem retrocesso.

Não-terminal	Alternativa	FIRST
expr	term R	{ (, num }
R	+ term R	{ + }
R	- term R	{ - }
R	ε	{ ε }
term	fact T	{ (, num }
T	* fact T	{ * }
T	/ fact T	{ / }
T	ε	{ ε }
fact	(expr)	{ (}
fact	num	{ num }

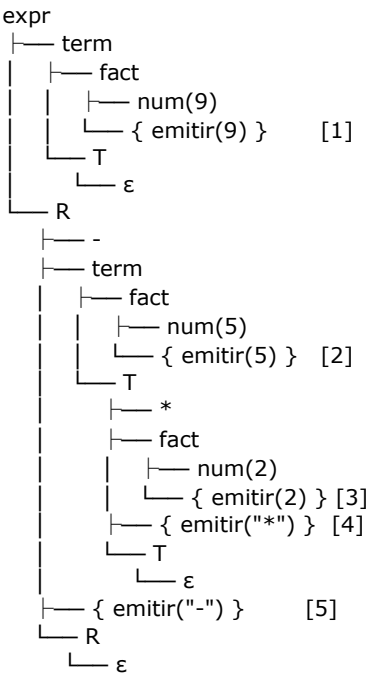
As alternativas não vazias de cada não-terminal têm conjuntos FIRST disjuntos. Para justificar também as alternativas vazias, usamos FOLLOW(R) = {), $ } e FOLLOW(T) = { +, -,), $ }, sendo $ o final da entrada. Esses FOLLOW não intersectam os FIRST das alternativas não vazias correspondentes. Assim, um lookahead de um token determina sem conflito a alternativa a aplicar, inclusive quando escolher ε.

4. Árvores de derivação e ordem das ações semânticas

Enunciado: Desenhe as árvores de derivação de 9-5*2 e (9-5)*2, com ações semânticas como filhos extras; mostre sua ordem de execução em percurso em profundidade e compare.

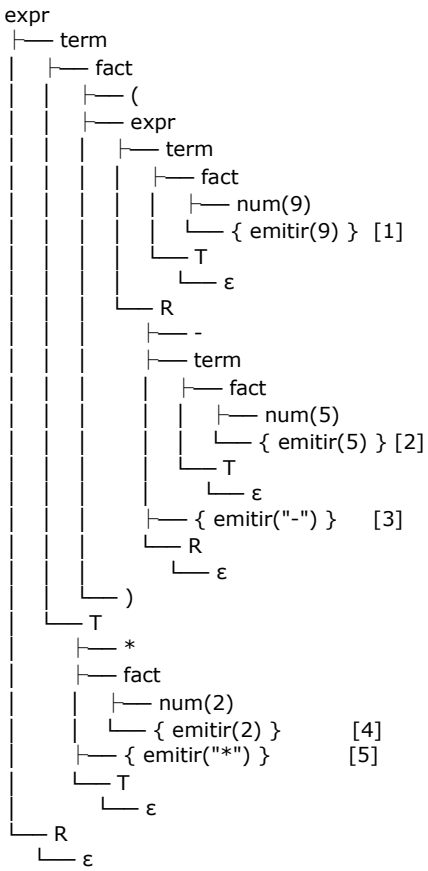
As árvores abaixo representam cada símbolo de uma produção como filho de seu não-terminal, inclusive as ações de emissão na posição em que são executadas. Os números [1] a [5] indicam a ordem das ações. A ordem é da esquerda para a direita em busca em profundidade.

4.1 Expressão: 9-5*2



Ordem: emitir(9) → emitir(5) → emitir(2) → emitir(*) → emitir(-). Saída: 9 5 2 * -. A multiplicação é traduzida antes da subtração, porque o term à direita do sinal de menos reconhece 5*2 por completo.

4.2 Expressão: (9-5)*2



Ordem: emitir(9) → emitir(5) → emitir(-) → emitir(2) → emitir(*). Saída: 9 5 - 2 *. Os parênteses levam o primeiro fact a reconhecer uma expr interna completa antes de permitir a multiplicação externa.

Comparação: sem parênteses, a expressão é 9 - (5 * 2); com parênteses, é (9 - 5) * 2. Como os agrupamentos mudam, muda também a ordem das ações e a notação pós-fixada.

Decisões de implementação e limitações

Decisões de implementação: O tradutor foi implementado em Python, utilizando apenas a biblioteca padrão. A análise sintática é realizada manualmente por procedimentos correspondentes aos componentes da gramática, com o auxílio das funções proximo() e match() para reconhecer os elementos da entrada e verificar se correspondem ao esperado.

A função proximo() ignora espaços e tabulações, reconhece números com vários dígitos como um único token e identifica os operadores e parênteses aceitos. As funções expr(), term() e fact() organizam a análise de acordo com a precedência dos operadores. As funções auxiliares R() e T() processam, respectivamente, as operações de adição e subtração e as operações de multiplicação e divisão.

A saída é construída durante a análise sintática, adicionando cada operador após o reconhecimento de seus operandos, conforme a notação pós-fixa. O programa permite processar várias expressões, uma por linha, tanto pela entrada padrão quanto por um arquivo informado como argumento. A função main() seleciona a origem da entrada, enquanto processar_entrada() percorre as linhas, ignora as vazias e encaminha cada expressão para tradução.

Limitações conhecidas: O tradutor aceita números inteiros não negativos, os operadores +, -, * e /, e parênteses. Não contempla números decimais, operadores unários, variáveis ou outros operadores além dos previstos na gramática. O programa gera a representação pós-fixa, mas não calcula o resultado numérico da expressão.

Os testes realizados verificam a precedência e a associatividade dos operadores, o uso de parênteses, inclusive aninhados, e o reconhecimento de números com vários dígitos. A implementação também prevê o tratamento de expressões inválidas, informando o erro na saída de erros e continuando o processamento das próximas linhas.

Declaração de uso de inteligência artificial

Foi utilizada a ferramenta ChatGPT como apoio didático para estudar os conceitos de gramáticas, eliminação de recursão à esquerda, ações semânticas, FIRST/FOLLOW e árvores de derivação; revisar respostas durante a preparação; e organizar/formatar o arquivo.